Freelance Marketplace Ecosystem

Database Project Report

Project 3 Mini Upwork
INT1313 Databases
Semester 1 2026 2027
Posts and Telecommunications Institute of Technology

Team G3

Member	Student email	GitHub
Huynh Thien Duc	n25dcat073@student.ptithcm.edu.vn	htduc1701
Pham Viet Hoang	n25dcat076@student.ptithcm.edu.vn	shinmagicc
Dang Quang Huy	n25dcat079@student.ptithcm.edu.vn	Kurookani

Submission version prepared on 20 October 2026

This report presents a relational database and a local application demonstration for hiring freelancers, agreeing contracts, approving milestones, recording payment attempts and aggregating client ratings. It extends the existing G3 implementation with normalized payment attempts and contract reviews while retaining the original account, proposal and delivery model.

The revised installation contains 12 base tables and 7,343 fictional records. Fresh execution on MySQL 26.7.0 completed 59 verification checks with 0 failures and executed all 15 reporting queries. These results describe the revised package; historical results from the earlier design are not reused.

Contents

Contents 2
1 Introduction and requirements 4
1 1 Problem statement and scope 4
1 2 User requirements 4
1 3 Functional requirements 4
1 4 Non functional requirements 5
1 5 Business rules 5
2 Database design 7
2 1 Conceptual model and notation 7
2 2 Accounts and expertise 8
2 3 Hiring and contract mapping 8
2 4 Work payments and ratings 9
2 5 Logical schema and physical choices 10
3 Normalization verification 12
3 1 Functional dependencies and candidate keys 12
3 2 Proof through BCNF 12
3 3 Repair of payment attempt redundancy 13
4 Data dictionary 14
4 1 users 14
4 2 clients 14
4 3 freelancers 14
4 4 skills 15
4 5 freelancer skills 15
4 6 job categories 15
4 7 jobs 15
4 8 proposals 16
4 9 contracts 16
4 10 milestones 17
4 11 payments 17
4 12 contract reviews 17
5 Implementation and transaction control 18
5 1 Budget protection and atomic writes 18
5 2 Views and indexes 18
5 3 Fixture coverage 19
6 Advanced queries and measured execution 20
Q 01 Jobs and proposal counts 20
Q 02 Freelancers with at least two proposals 21

Q 03 Jobs with no proposals 21
Q 04 Rank freelancer contract value 21
Q 05 Client successful payment totals 21
Q 06 Contracts not fully paid 22
Q 07 Completed work not yet paid 22
Q 08 Jobs above mean maximum budget 22
Q 09 Freelancers with at least three skills 23
Q 10 Contract milestone payment report 23
Q 11 Clients posting the most jobs 23
Q 12 Strictly overdue unfinished milestones as of execution date 24
Q 13 Rated freelancers ordered by average and review count 24
Q 14 Review distribution by star value 24
Q 15 Successful revenue compared with rated contract count without fan-out 24
7 Verification and security 25
7 1 Execution environment and scope 25
7 2 Test results 25
7 3 Roles and actor identity 27
7 4 Security and deployment limits 27
8 Application integration and group defense 28
8 1 Running the local demo 28
8 2 Demonstration sequence 28
8 3 Defense and peer evaluation 28
9 Conclusion and limitations 29
References 30
Appendix A Business rule traceability 31
Appendix B Reproduction and artifact map 32

1 Introduction and requirements

1 1 Problem statement and scope

Clients need a consistent way to publish jobs, compare proposals and release payment for accepted work. Freelancers need to submit offers, confirm selected work and track delivery. The database must prevent a client selection from becoming a binding agreement before the freelancer confirms, and must connect every payment to approved work and the correct parties.

The supplied project scenario also requires multi-party transactions, milestone tracking and user-rating aggregation. The revised scope includes a simple client-to-freelancer review after a completed contract. Escrow custody, external payment gateways, refunds, currency conversion, dispute resolution and a production identity platform remain outside this coursework implementation. Payment outcomes are simulated records rather than bank transfers.

The two supplied teaching documents define the reporting structure and assessment deliverables. MySQL, the one-way rating policy, a single USD demo currency, the exact business rules and the size of the generated fixture are team design decisions. The PDFs do not prescribe 11 tables or a minimum of 1,000 rows. The report follows selected documentation principles; it does not claim ISO certification.

1 2 User requirements

ID	User need
UR-01	Clients publish work, select an offer, allocate and approve milestones, close paid contracts and review freelancers.
UR-02	Freelancers maintain skills, submit proposals, confirm selected offers and mark their assigned work in progress or completed.
UR-03	A trusted payment operator records a simulated provider outcome for the owning client and approved milestone.
UR-04	Reporting users inspect proposal counts, financial summaries and ratings without reading password hashes.

1 3 Functional requirements

ID	System capability	Rules
FR-01	Register exactly one typed account and matching subtype	BR-01 02
FR-02	Maintain profiles and freelancer skill associations	BR-05
FR-03	Post jobs with required client and category	BR-03 04
FR-04	Submit one proposal per freelancer and job	BR-06 to 09
FR-05	Select one proposal and require freelancer confirmation	BR-10 11
FR-06	Create one matching contract atomically	BR-11 12
FR-07	Allocate positive milestones within contract value and dates	BR-13 14
FR-08	Progress work and require client approval	BR-15
FR-09	Record payment attempts and allow at most one success	BR-16
FR-10	Complete only fully allocated approved and paid contracts	BR-14 to 17
FR-11	Submit one client review after contract completion	BR-17
FR-12	Aggregate ratings with review counts and NULL for no ratings	BR-18

Profiles and skills are supported at the data model and controlled administration level. The small application demonstrates the hiring and delivery workflow; it does not implement every profile administration screen.

1 4 Non functional requirements

ID	Target	Verification or limit
NFR-01	Transactional integrity under READ COMMITTED	Constraints, workflow tests, two-session budget test and full-dataset invariants.
NFR-02	Actor procedures and least privilege	Direct table DML and password-hash access denied; authenticated native username checked against actor.
NFR-03	Local warm reporting queries below 200 ms	A project target, measured on the revised fixture using five repetitions; not a production SLA.
NFR-04	TLS for remote access and encryption at rest	Deployment requirements; application binds to loopback and does not certify at-rest encryption.
NFR-05	Daily backup with RPO 24 h and RTO 4 h	Operational targets; restore drill and availability test not performed.
NFR-06	Reproducible installation and readable SQL	Fresh install, explicit modules, UTF-8, lowercase snake_case and uppercase keywords.

1 5 Business rules

BR-01: Each USER must have a unique user_id.

BR-02: In this version, a USER is specialized into CLIENT or FREELANCER; the two subtypes are distinct and disjoint.

BR-03: Each CLIENT can post many JOB records, but each JOB belongs to one CLIENT.

BR-04: Each JOB belongs to one JOB_CATEGORY, while one JOB_CATEGORY can contain many JOB records.

BR-05: A FREELANCER can have many SKILL records and a SKILL can belong to many FREELANCER records; the relationship is resolved through FREELANCER_SKILL.

BR-06: A JOB can receive many PROPOSAL records.

BR-07: A FREELANCER can submit many PROPOSAL records for different JOB records.

BR-08: A FREELANCER may not submit more than one PROPOSAL for the same JOB.

BR-09: Each PROPOSAL belongs to exactly one JOB and one FREELANCER.

BR-10: A JOB may have at most one PROPOSAL selected (offered) by the CLIENT at a time.

BR-11: An accepted PROPOSAL requires mutual consent: it must be selected by the CLIENT and subsequently confirmed by the FREELANCER to generate exactly one CONTRACT. A proposal that lacks this final confirmation from the freelancer does not create a CONTRACT.

BR-12: Each CONTRACT involves exactly one CLIENT and one FREELANCER.

BR-13: A CONTRACT can be divided into multiple MILESTONE records; each MILESTONE belongs to one CONTRACT.

BR-14: Each MILESTONE amount must be greater than 0, and the total milestone value must not exceed the CONTRACT value.

BR-15: A MILESTONE may trigger a PAYMENT only after its work status is marked as COMPLETED and approved by the CLIENT.

BR-16: Each payment attempt references one approved milestone. The payer, payee and amount are derived through its immutable contract and milestone terms; only one attempt per milestone may be PAID.

BR-17: Only the owning client may submit one immutable review of a COMPLETED contract, using an integer score from 1 to 5. The recipient is the contracted freelancer.

BR-18: A freelancer rating is the arithmetic mean of that freelancer's contract review scores. Show the count alongside the mean; an unrated freelancer has count 0 and a NULL mean.

BR-01 to BR-15 preserve the original G3 policy. BR-16 has been restated to preserve the transfer information without storing it repeatedly in every attempt. BR-17 and BR-18 make the rating challenge explicit. These policies are design choices that implement the scenario; they are not quoted as additional instructions from the lecturer.

2 Database design

2 1 Conceptual model and notation

The model uses UML-style class compartments and association multiplicities, one of the industry notation options permitted by the documentation handout. A label 1 means exactly one, 0..1 means optional one and 0..* means zero or many. Each endpoint label describes how many instances at that endpoint can be associated with one instance at the other endpoint. Crossings without junctions are not additional relationships. PK, FK and UQ identify physical keys; they supplement the conceptual associations.

USER has total, disjoint specialization into CLIENT and FREELANCER. The separate optional subtype links must be read together with that invariant: every user belongs to exactly one subtype, selected by user_type. Shared primary keys preserve identity; insertion, type and retention triggers enforce the rule.

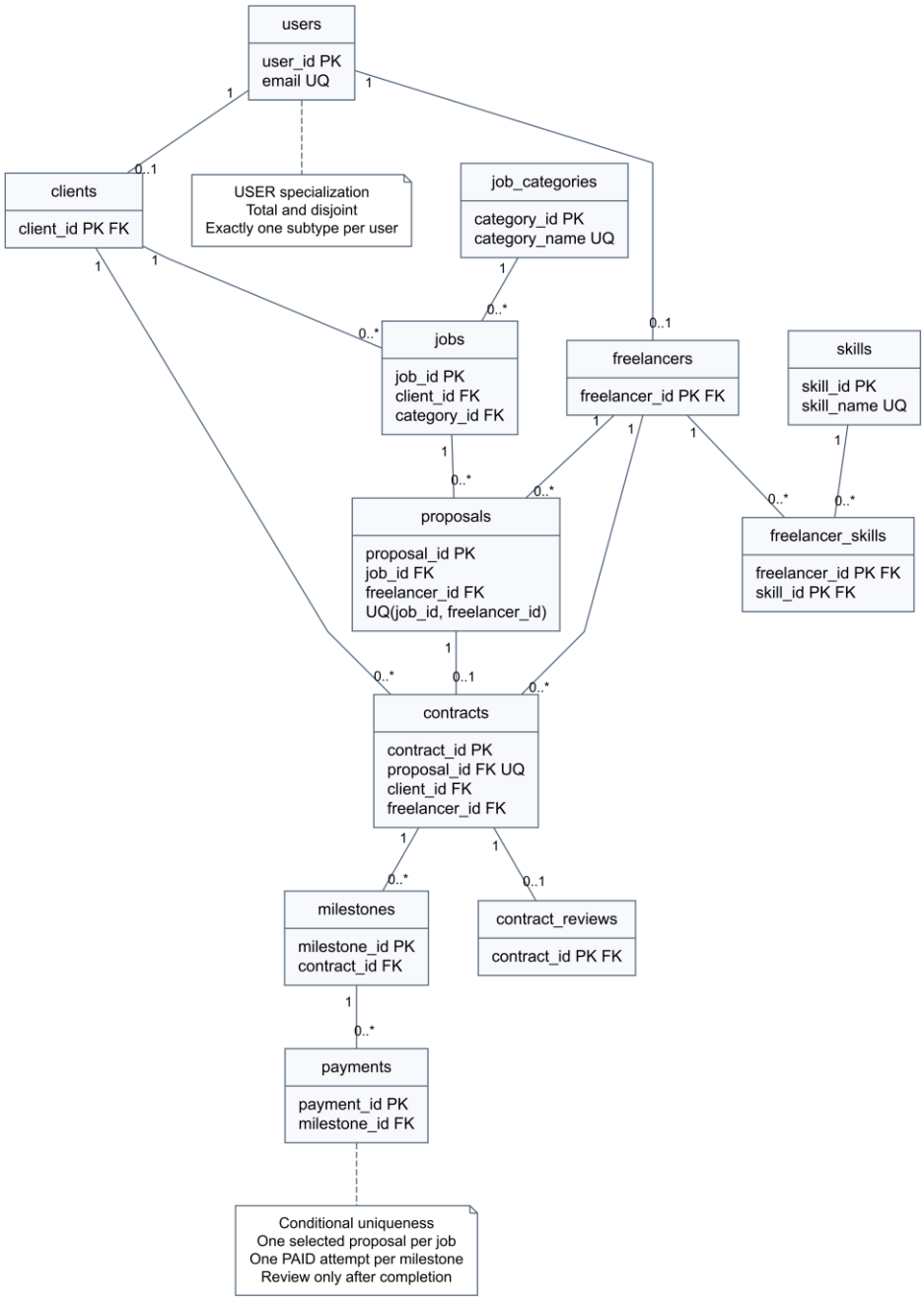


Figure 1 Unified model showing all twelve business entities and their key relationships

2 2 Accounts and expertise

A freelancer can have no skills or several skills. A skill can be shared by several freelancers. freelancer_skills resolves that many-to-many association and stores the level for the whole freelancer-skill pair. Email, skill name and category name are alternate keys under the configured case-insensitive collation; phone, company name and professional title are not unique.

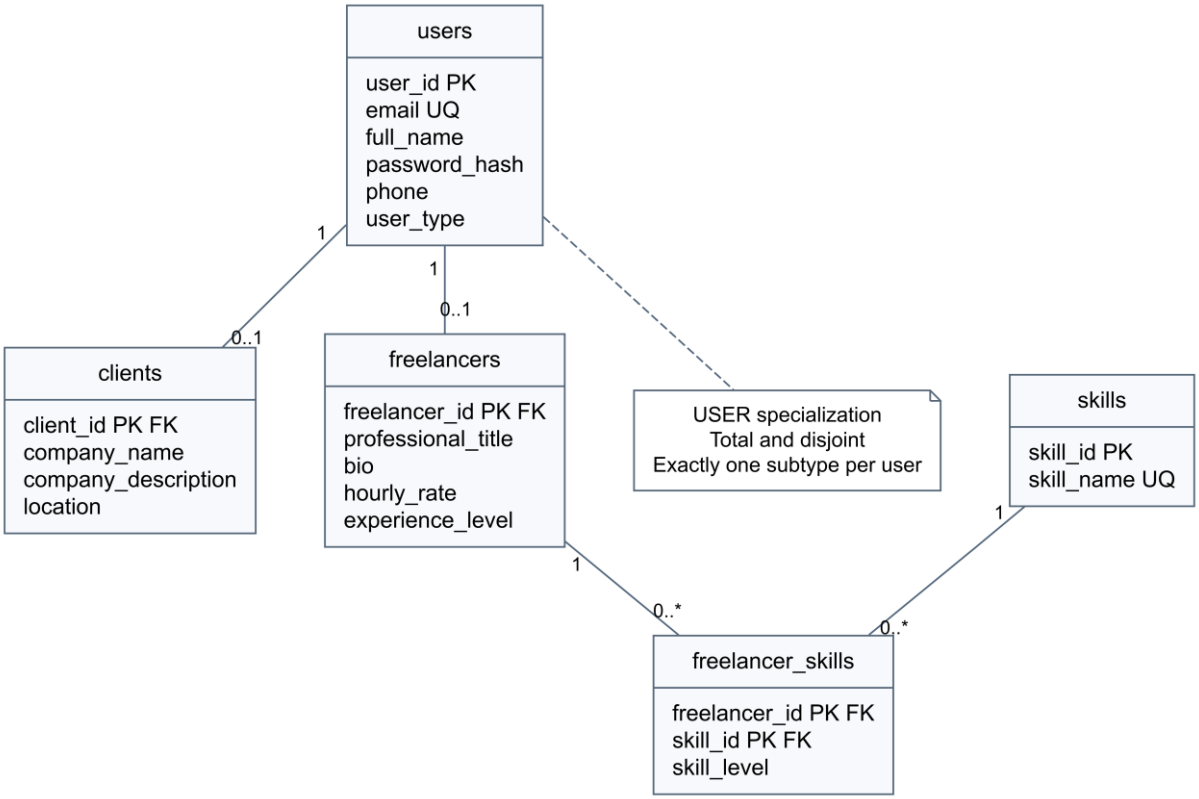


Figure 2 Account specialization and the freelancer skill association

2 3 Hiring and contract mapping

Each job references exactly one client and category. Each proposal references one job and freelancer; the pair is unique. A job can receive many proposals but can have at most one selected or confirmed proposal at a time. A proposal can have zero or one contract; every contract must reference one confirmed proposal. A UNIQUE NOT NULL proposal_id enforces the optional one-to-one mapping from proposal to contract.

The PENDING to CLIENT_ACCEPTED transition is an offer. The assigned freelancer then changes it to CONFIRMED through the appropriate procedure. A trigger creates the contract in the same transaction, validates the parties and amount, and moves the job to IN_PROGRESS. If contract creation fails, the triggering update is rolled back. Completed or cancelled agreements remain associated with the original selected proposal; a new hiring round needs a new job.

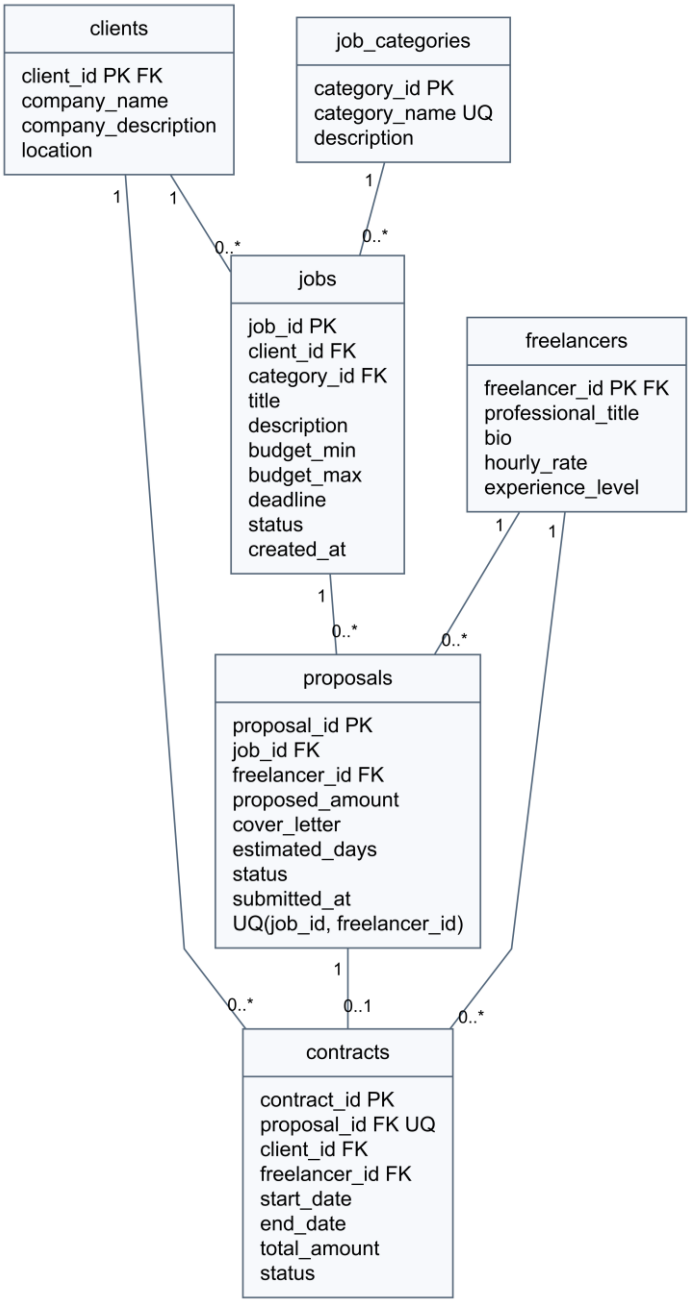


Figure 3 Job proposals and contract formation

2 4 Work payments and ratings

A new contract can initially have no milestones. Each milestone belongs to exactly one contract and moves through PENDING, IN_PROGRESS, COMPLETED and APPROVED. Completion by a freelancer and approval by a client are separate actions. The sum of milestone amounts cannot exceed the fixed contract amount. A contract can complete only when the full amount is allocated, every milestone is approved and each has a successful payment.

Each payment row records an attempt for one milestone. Failed attempts are retained, and a pending attempt can reach a terminal state. The functional unique index allows multiple non-successful attempts but at most one PAID attempt per milestone. v_payment_details derives payer_id, payee_id and amount from the milestone and immutable contract parties. Started milestone amounts and identities cannot change, so retries keep the same transfer terms without repeating them in payments.

contract_reviews has one row per completed contract and stores an integer score, optional comment and timestamp. The owning client is the reviewer and the contract freelancer is the recipient, so neither identity is repeated in the review row. This is an explicit one-way review policy: freelancers do not rate clients. Because account subtypes are disjoint and the contract binds both parties, the reviewer cannot be the recipient. Reviews are immutable and retained. The mean is unweighted across reviewed contracts; unreviewed contracts do not contribute a score of zero.

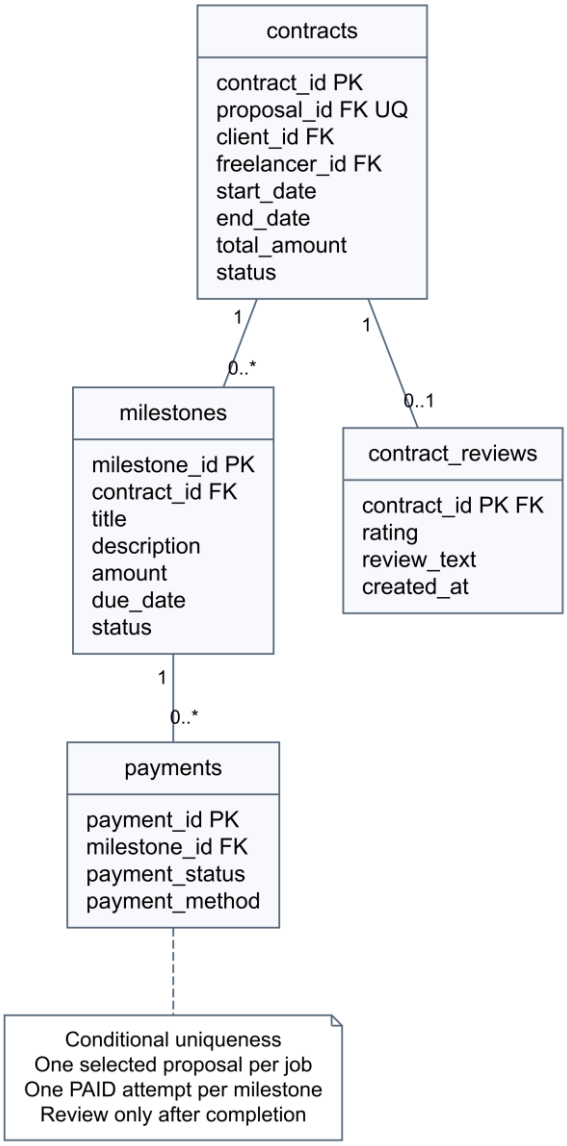


Figure 4 Milestone delivery payment attempts and contract reviews

2 5 Logical schema and physical choices

users(user_id, email, full_name, password_hash, phone, user_type)

clients(client_id, company_name, company_description, location)

freelancers(freelancer_id, professional_title, bio, hourly_rate, experience_level)

skills(skill_id, skill_name)

freelancer_skills(freelancer_id, skill_id, skill_level)

job_categories(category_id, category_name, description)

jobs(job_id, client_id, category_id, title, description, budget_min, budget_max, deadline, status, created_at)

proposals(proposal_id, job_id, freelancer_id, proposed_amount, cover_letter, estimated_days, status, submitted_at)

contracts(contract_id, proposal_id, client_id, freelancer_id, start_date, end_date, total_amount, status)

milestones(milestone_id, contract_id, title, description, amount, due_date, status)

payments(payment_id, milestone_id, payment_status, payment_method)

contract_reviews(contract_id, rating, review_text, created_at)

All relationships use explicit foreign keys with restrictive parent deletion. Integer identities are auto-incremented where declared. Exact DECIMAL values avoid floating-point rounding for amounts. VARCHAR status domains are constrained by named CHECK expressions. UTF-8 supports the fictional Vietnamese fixture. Profiles and skill associations can be empty before work begins.

```
UNIQUE KEY uq_selected_job
  ((CASE WHEN status IN ('CLIENT_ACCEPTED','CONFIRMED')
         THEN job_id ELSE NULL END))
UNIQUE KEY uq_paid_milestone
  ((CASE WHEN payment_status='PAID'
         THEN milestone_id ELSE NULL END))
```

Conditional uniqueness is expressed as functional indexes rather than visible generated helper attributes. NULL index results permit the non-selected proposals and failed or pending attempts. These indexes are implementation access structures, not independently stored business facts in the logical schema. The SQL uses features available in MySQL 8.0.16 and later; this execution was verified specifically on MySQL 26.7.0, not on an 8.0 server.

3 Normalization verification

3 1 Functional dependencies and candidate keys

We analyze each declared business relation using the following FD covers. “All” means all other business attributes in that relation; FDs do not treat a surrogate primary key as the only possible determinant. The analysis assumes no hidden dependency such as a freelancer determining a proposal amount, or a phone number uniquely identifying a user. Currency is a project-wide constant rather than a stored multi-currency attribute.

Relation	Candidate keys	FD cover
users	user_id; email	user_id → all; email → all
clients	client_id	client_id → all
freelancers	freelancer_id	freelancer_id → all
skills	skill_id; skill_name	skill_id → skill_name; skill_name → skill_id
freelancer_skills	(freelancer_id, skill_id)	(freelancer_id, skill_id) → skill_level
job_categories	category_id; category_name	each candidate key → all
jobs	job_id	job_id → all
proposals	proposal_id; (job_id, freelancer_id)	each candidate key → all
contracts	contract_id; proposal_id	each candidate key → all
milestones	milestone_id	milestone_id → all
payments	payment_id	payment_id → milestone_id, payment_status, payment_method
contract_reviews	contract_id	contract_id → rating, review_text, created_at

3 2 Proof through BCNF

1NF holds because each business cell has one scalar value, tuples are identified by keys and skills are represented by separate association rows. Text descriptions are scalar content, not comma-separated identifiers or repeating attribute groups.

For 2NF, the composite freelancer-skill key is minimal: a freelancer can have several skills and a skill can belong to several freelancers. Neither component determines skill_level. The alternate proposal key (job_id, freelancer_id) is minimal because jobs receive multiple proposals and freelancers submit to multiple jobs. Its non-prime attributes describe that complete offer; no proper subset determines those attributes. All other candidate keys in the stated cover are single attributes, so no partial dependency on a proper nonempty subset exists.

For each listed candidate key K, K+ contains every relation attribute by the stated cover. The only nontrivial determinants in that cover are these candidate keys. Dependencies inferred by transitivity or augmentation do not introduce an independent non-key business determinant under the assumptions. Every relation therefore satisfies BCNF, and consequently 3NF, at the declared business-schema level. Column domains and equality collisions under the chosen collation are part of those assumptions; the proof is not a claim about arbitrary undocumented domain rules.

contracts repeats parties available via proposal and job, but proposal_id is itself a candidate key of contracts. Thus the within-relation dependency proposal_id → client_id, freelancer_id is not a 3NF violation. Cross-table consistency is still enforced by validation and identity immutability. This distinction avoids confusing redundant join paths with a non-key determinant inside a relation.

3 3 Repair of payment attempt redundancy

In the previous design, a milestone could have multiple attempts and milestone_id determined the repeated amount and participants. milestone_id was not a superkey of payments, so those non-prime attributes violated 3NF. The revised payments relation retains only attempt-specific facts and the milestone foreign key. It has payment_id as its sole candidate key under the stated dependencies.

The original extended payment representation is recovered losslessly by joining payments to milestones on milestone_id and then contracts on contract_id. Each join targets a declared primary key and follows a NOT NULL foreign key, so each attempt produces exactly one extended row: no attempt is lost or duplicated. No independent amount or participant value is discarded because the old guards required equality with those parent terms. This is a reconstruction argument across the existing parent relations, not a claim that a projection of payments alone contains all transfer information.

Approval, one successful payment, participant matching and budget limits remain cross-row or cross-relation rules. Normal forms do not establish these rules; foreign keys, conditional uniqueness, triggers and authenticated procedures enforce them. The fixed parties and frozen started milestone amount make the derived financial record stable within this coursework scope. Refunds, currency conversion and independent gateway settlement amounts would require a separate model.

4 Data dictionary

The following tables are generated from the executed schema metadata and supplemented with business meanings. They cover every base table. PK and UQ denote primary and alternate unique keys; FK denotes a foreign key. “None” means no explicit default for a required value. An auto-increment identity is not an ordinary application default. Optional columns accept NULL. All parent references use RESTRICT or equivalent NO ACTION.

4 1 users

Attribute	Data type	Key	Null	Default	Meaning and rules
user_id	int	PK	NO	Identity	Account identifier
email	varchar(150)	UQ	NO	None	Unique account email
full_name	varchar(100)	—	NO	None	Display name
password_hash	varchar(255)	—	NO	None	Credential hash; fixture values cannot log in
phone	varchar(20)	—	YES	NULL	Optional contact number
user_type	varchar(20)	—	NO	None	CLIENT or FREELANCER; immutable

ck_users_type: (`user_type` in (_utf8mb4\'CLIENT\',_utf8mb4\'FREELANCER\'))

4 2 clients

Attribute	Data type	Key	Null	Default	Meaning and rules
client_id	int	FK, PK	NO	None	Hiring account identifier; References users.user_id
company_name	varchar(150)	—	YES	NULL	Optional company display name
company_description	text	—	YES	NULL	Optional company description
location	varchar(150)	—	YES	NULL	Optional profile location

4 3 freelancers

Attribute	Data type	Key	Null	Default	Meaning and rules
freelancer_id	int	FK, PK	NO	None	Freelancer account identifier; References users.user_id
professional_title	varchar(150)	—	NO	Not specified	Professional profile title
bio	text	—	YES	NULL	Optional profile biography
hourly_rate	decimal(10,2)	—	NO	0.00	Nonnegative advertised USD rate
experience_level	varchar(20)	—	NO	INTERMEDIATE	BEGINNER, INTERMEDIATE, ADVANCED or EXPERT

ck_freelancer_rate: (`hourly_rate` >= 0)

ck_freelancer_level: (`experience_level` in (_utf8mb4\'BEGINNER\',_utf8mb4\'INTERMEDIATE\',_utf8mb4\'ADVANCED\',_utf8mb4\'EXPERT\'))

4 4 skills

Attribute	Data type	Key	Null	Default	Meaning and rules
skill_id	int	PK	NO	Identity	Skill identifier
skill_name	varchar(100)	UQ	NO	None	Unique skill name

4 5 freelancer skills

Attribute	Data type	Key	Null	Default	Meaning and rules
freelancer_id	int	FK, PK	NO	None	Freelancer account identifier; References freelancers.freelancer_id
skill_id	int	FK, PK	NO	None	Skill identifier; References skills.skill_id
skill_level	varchar(20)	—	NO	INTERMEDIATE	Proficiency of this freelancer in this skill

ck_fs_level: (`skill_level` in (_utf8mb4\'BEGINNER\',_utf8mb4\'INTERMEDIATE\',_utf8mb4\'ADVANCED\',_utf8mb4\'EXPERT\'))

4 6 job categories

Attribute	Data type	Key	Null	Default	Meaning and rules
category_id	int	PK	NO	Identity	Job category identifier
category_name	varchar(100)	UQ	NO	None	Unique category name
description	text	—	YES	NULL	Business description

4 7 jobs

Attribute	Data type	Key	Null	Default	Meaning and rules
job_id	int	PK	NO	Identity	Job identifier
client_id	int	FK	NO	None	Hiring account identifier; References clients.client_id
category_id	int	FK	NO	None	Job category identifier; References job_categories.category_id
title	varchar(200)	—	NO	None	Job or milestone title
description	text	—	NO	None	Business description
budget_min	decimal(15,2)	—	NO	0.00	Nonnegative lower budget
budget_max	decimal(15,2)	—	NO	0.00	Upper budget at least budget_min
deadline	date	—	YES	NULL	Optional date later than DATE(created_at)
status	varchar(20)	—	NO	OPEN	Constrained workflow state
created_at	datetime	—	NO	Current timestamp	Creation timestamp

ck_jobs_budget: ((`budget_min` >= 0) and (`budget_max` >= `budget_min`))

ck_jobs_dates: ((`deadline` is null) or (`deadline` > cast(`created_at` as date)))

ck_jobs_status: (`status` in (_utf8mb4\'OPEN\',_utf8mb4\'IN_PROGRESS\',_utf8mb4\'COMPLETED\',_utf8mb4\'CANCELLED\'))

4 8 proposals

Attribute	Data type	Key	Null	Default	Meaning and rules
proposal_id	int	PK	NO	Identity	Offer identifier
job_id	int	FK, UQ	NO	None	Job identifier; References jobs.job_id
freelancer_id	int	FK, UQ	NO	None	Freelancer account identifier; References freelancers.freelancer_id
proposed_amount	decimal(15,2)	—	NO	None	Strictly positive quoted USD amount
cover_letter	text	—	NO	None	Proposal narrative
estimated_days	int	—	NO	None	Strictly positive delivery duration
status	varchar(20)	—	NO	PENDING	Constrained workflow state
submitted_at	datetime	—	NO	Current timestamp	Offer submission timestamp

ck_proposal_amount: (`proposed_amount` > 0)

ck_proposal_days: (`estimated_days` > 0)

ck_proposal_status: (`status` in (_utf8mb4\'PENDING\',_utf8mb4\'CLIENT_ACCEPTED\',_utf8mb4\'CONFIRMED\',_utf8mb4\'REJECTED\',_utf8mb4\'WITHDRAWN\'))

Additional uniqueness: (job_id, freelancer_id) is a composite alternate key. The functional unique index reserves a job only for CLIENT_ACCEPTED or CONFIRMED proposals.

4 9 contracts

Attribute	Data type	Key	Null	Default	Meaning and rules
contract_id	int	PK	NO	Identity	Contract identifier
proposal_id	int	FK, UQ	NO	None	Offer identifier; References proposals.proposal_id
client_id	int	FK	NO	None	Hiring account identifier; References clients.client_id
freelancer_id	int	FK	NO	None	Freelancer account identifier; References freelancers.freelancer_id
start_date	date	—	NO	None	Fixed agreement start date
end_date	date	—	YES	NULL	Optional end after start; contains milestone dates
total_amount	decimal(15,2)	—	NO	None	Fixed positive agreement amount
status	varchar(20)	—	NO	ACTIVE	Constrained workflow state

ck_contract_amount: (`total_amount` > 0)

ck_contract_dates: ((`end_date` is null) or (`end_date` > `start_date`))

ck_contract_status: (`status` in (_utf8mb4\'ACTIVE\',_utf8mb4\'COMPLETED\',_utf8mb4\'CANCELLED\'))

4 10 milestones

Attribute	Data type	Key	Null	Default	Meaning and rules
milestone_id	int	PK	NO	Identity	Delivery milestone identifier
contract_id	int	FK	NO	None	Contract identifier; References contracts.contract_id
title	varchar(200)	—	NO	None	Job or milestone title
description	text	—	YES	NULL	Business description
amount	decimal(15,2)	—	NO	None	Positive USD milestone amount; parent budget guard
due_date	date	—	YES	NULL	Optional deadline within contract dates
status	varchar(20)	—	NO	PENDING	Constrained workflow state

ck_milestone_amount: (`amount` > 0)

ck_milestone_status: (`status` in (_utf8mb4\'PENDING\',_utf8mb4\'IN_PROGRESS\',_utf8mb4\'COMPLETED\',_utf8mb4\'APPROVED\'))

4 11 payments

Attribute	Data type	Key	Null	Default	Meaning and rules
payment_id	int	PK	NO	Identity	Payment attempt identifier
milestone_id	int	FK	NO	None	Delivery milestone identifier; References milestones.milestone_id
payment_status	varchar(20)	—	NO	PENDING	PENDING, PAID or FAILED
payment_method	varchar(50)	—	NO	None	PAYPAL, CREDIT_CARD or BANK_TRANSFER

ck_payment_status: (`payment_status` in (_utf8mb4\'PENDING\',_utf8mb4\'PAID\',_utf8mb4\'FAILED\'))

ck_payment_method: (`payment_method` in (_utf8mb4\'PAYPAL\',_utf8mb4\'CREDIT_CARD\',_utf8mb4\'BANK_TRANSFER\'))

The unique functional index permits at most one PAID attempt per milestone. Transfer amount and parties are exposed by v_payment_details and are not columns of payments.

4 12 contract reviews

Attribute	Data type	Key	Null	Default	Meaning and rules
contract_id	int	FK, PK	NO	None	Contract identifier; References contracts.contract_id
rating	tinyint	—	NO	None	Integer 1 to 5; client review of completed contract
review_text	varchar(1000)	—	YES	NULL	Optional review comment
created_at	datetime	—	NO	Current timestamp	Creation timestamp

ck_review_rating: (`rating` between 1 and 5)

contract_id is both the primary key and a foreign key. One client review per completed contract is enforced without repeating reviewer or recipient identifiers.

5 Implementation and transaction control

Module	Purpose
01_schema.sql	Twelve tables, keys, CHECK domains and conditional unique indexes
02_triggers.sql	25 triggers for subtypes, agreements, budgets, payments, review retention and job synchronization
03_views_indexes.sql	Five views and two explicit workload indexes
04_procedures.sql	9 authenticated workflow procedures
05_seed.sql	Small inherited fictional workflow fixture
08_seed_1000_jobs.sql	Retained larger fictional fixture with normalized payments
10_seed_reviews.sql	Complete eligible contracts and seed 120 fictional reviews
07_security.sql	Roles, locked sample native accounts, GRANT and REVOKE
09_verify_counts.sql	Counts for all twelve tables
06_queries.sql	Fifteen business reporting queries

5 1 Budget protection and atomic writes

A row CHECK cannot enforce SUM(milestone.amount) across a contract. Both milestone insert and update triggers lock the contract row before calculating the current allocation. Supported write procedures explicitly use READ COMMITTED and standalone transactions. The parent row serializes competing allocations. The tested race issued two simultaneous amounts of 60 against a contract value of 100: one succeeded and one was rejected, leaving a total of 60.

Confirmation locks the job before changing the proposal; the unique reservation index is a second defense against duplicate selections. Payment procedures lock the relevant contract and milestone. Review submission locks the completed contract. The procedures roll back and resignal on SQL errors. They must not be called inside a caller-managed transaction because they establish their own transaction characteristics and boundaries. Arbitrary bulk INSERT SELECT paths that read a parent also locked by a child trigger are not the supported write interface; the seed uses VALUES and a cursor for review creation.

5 2 Views and indexes

Object	Business meaning or rationale
v_job_proposals	Includes jobs with no proposals through LEFT JOIN.
v_payment_details	Reconstructs exact transfer parties and amount for each attempt.
v_contract_overview	Aggregates milestones and successful payment values separately to avoid fan-out.
v_client_spend	Counts only PAID attempts and retains zero-spend clients.
v_freelancer_ratings	Review count and arithmetic mean, including count 0 and NULL mean.
ix_jobs_status_budget	Supports open jobs filtered or ordered by budget; adds write and storage cost.
ix_milestones_status_due	Supports status and due-date filtering; a tiny table may still be scanned.

Foreign key and unique constraints supply additional supporting indexes. We do not add a redundant job_id-only index when the proposal alternate key already starts with job_id. Plans are recorded for all fifteen

queries. No index speedup percentage is claimed because a controlled before-and-after benchmark was not run for this revision.

5 3 Fixture coverage

Table	Fresh installation rows
users	248
clients	43
freelancers	205
skills	26
freelancer_skills	610
job_categories	11
jobs	1008
proposals	1811
contracts	605
milestones	1810
payments	846
contract_reviews	120
Total	7343

The fixture retains 1,008 distinct jobs and includes proposals waiting for selection or confirmation, active and completed contracts, all four milestone states, and failed, pending and successful payment attempts. The additional 120 rows are completed-contract reviews with scores 3 to 5; boundary values and rejection cases are tested separately. All people, businesses and transactions are fictional; email uses example.test and credential hashes are non-login placeholders. Sample native MySQL users are separate from business user rows.

6 Advanced queries and measured execution

The catalog below supplies JOIN, subquery, aggregation, CTE and window-function examples. Each statement is also runnable in database/sql/06_queries.sql. Output previews and EXPLAIN plans are recorded in evidence/query_results.json. Query execution occurs after verification-only rows have been inserted: therefore query row counts do not describe a pristine installation. Dataset_counts.json separately records the fresh 7,343-row fixture before tests.

Timings are median wall-clock milliseconds for five warm executions, including local client fetch. They are indicative measurements on this local instance, not a production capacity benchmark. SQL statements use the database server date. Q-12 now means strictly overdue, using due_date < CURRENT_DATE; it no longer mixes future due dates into an overdue label.

Query	Result rows	Median ms
Q-01	1010	11.699
Q-02	204	0.788
Q-03	101	1.062
Q-04	204	1.508
Q-05	43	37.493
Q-06	486	5.645
Q-07	483	2.447
Q-08	494	1.643
Q-09	202	1.667
Q-10	607	7.277
Q-11	40	0.433
Q-12	0	0.219
Q-13	41	1.140
Q-14	3	0.202
Q-15	205	2.785

Q 01 Jobs and proposal counts

Expected: One row for every job, including jobs with zero proposals.

```
SELECT *
FROM v_job_proposals
ORDER BY job_id;
```

Actual execution: 1010 rows; median 11.699 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

First row: job_id=1; client_id=1; title=Xây dựng cổng đặt hàng cho An Nhiên Retail; status=IN_PROGRESS; budget_max=1000.00; proposal_count=2; selected_count=1

Q 02 Freelancers with at least two proposals

Expected: Only freelancers with at least two proposals.

```
SELECT freelancer_id,
       COUNT(*) AS proposals
FROM proposals
GROUP BY freelancer_id
HAVING COUNT(*) >= 2
ORDER BY freelancer_id;
```

Actual execution: 204 rows; median 0.788 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

First row: freelancer_id=4; proposals=5

Q 03 Jobs with no proposals

Expected: Jobs with no matching proposal.

```
SELECT j.job_id,
       j.title
FROM jobs AS j
WHERE NOT EXISTS
        (SELECT 1
         FROM proposals AS p
         WHERE p.job_id = j.job_id)
ORDER BY j.job_id;
```

Actual execution: 101 rows; median 1.062 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

First row: job_id=7; title=Thiết kế trang giới thiệu bộ sưu tập An Nhiên Retail

Q 04 Rank freelancer contract value

Expected: Freelancers ranked by the sum of non-cancelled contract value.

```
WITH totals AS
    (SELECT freelancer_id,
            SUM(total_amount) AS value
     FROM contracts
     WHERE status <> 'CANCELLED'
     GROUP BY freelancer_id)
SELECT freelancer_id,
       value,
       DENSE_RANK() OVER (
                           ORDER BY value DESC) AS value_rank
FROM totals
ORDER BY value_rank,
         freelancer_id;
```

Actual execution: 204 rows; median 1.508 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

First row: freelancer_id=21159; value=10740.00; value_rank=1

Q 05 Client successful payment totals

Expected: Every client with PAID counts and amount, including zero spend.

```
SELECT *
FROM v_client_spend
ORDER BY client_id;
```

Actual execution: 43 rows; median 37.493 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

First row: client_id=1; paid_count=3; paid_amount=800.00

Q 06 Contracts not fully paid

Expected: Contracts whose successful payments are below their agreed value.

```
SELECT contract_id,
       total_amount,
       paid_amount,
       outstanding_amount
FROM v_contract_overview
WHERE outstanding_amount > 0
ORDER BY contract_id;
```

Actual execution: 486 rows; median 5.645 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

First row: contract_id=1; total_amount=1000.00; paid_amount=400.00; outstanding_amount=600.00

Q 07 Completed work not yet paid

Expected: Completed or approved milestones without a successful attempt.

```
SELECT m.milestone_id,
       m.status,
       m.amount
FROM milestones AS m
WHERE m.status IN ('COMPLETED',
                   'APPROVED')
    AND NOT EXISTS
        (SELECT 1
         FROM payments AS p
         WHERE p.milestone_id=m.milestone_id
             AND p.payment_status='PAID')
ORDER BY m.milestone_id;
```

Actual execution: 483 rows; median 2.447 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

First row: milestone_id=2; status=COMPLETED; amount=600.00

Q 08 Jobs above mean maximum budget

Expected: Jobs above the mean maximum budget.

```
SELECT job_id,
       budget_max
FROM jobs
WHERE budget_max >
        (SELECT AVG(budget_max)
         FROM jobs)
ORDER BY job_id;
```

Actual execution: 494 rows; median 1.643 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

First row: job_id=22002; budget_max=2400.00

Q 09 Freelancers with at least three skills

Expected: Freelancers with three or more assigned skills.

```
SELECT f.freelancer_id,
       f.professional_title,
       COUNT(fs.skill_id) AS skill_count
FROM freelancers AS f
INNER JOIN freelancer_skills AS fs ON fs.freelancer_id=f.freelancer_id
GROUP BY f.freelancer_id,
         f.professional_title
HAVING COUNT(fs.skill_id)>=3
ORDER BY f.freelancer_id;
```

Actual execution: 202 rows; median 1.667 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

First row: freelancer_id=4; professional_title=Lập trình viên Python và cơ sở dữ liệu; skill_count=3

Q 10 Contract milestone payment report

Expected: One row per contract without payment retry fan-out.

```
SELECT *
FROM v_contract_overview
ORDER BY contract_id;
```

Actual execution: 607 rows; median 7.277 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

First row: contract_id=1; client_id=1; freelancer_id=4; total_amount=1000.00; status=ACTIVE; milestone_count=2; allocated_amount=1000.00; paid_amount=400.00; outstanding_amount=600.00

Q 11 Clients posting the most jobs

Expected: All clients tied for the greatest job count.

```
WITH counts AS
    (SELECT client_id,
            COUNT(*) AS job_count
     FROM jobs
     GROUP BY client_id),
     ranked AS
    (SELECT *,
            DENSE_RANK() OVER (
                               ORDER BY job_count DESC) AS r
     FROM counts)
SELECT client_id,
       job_count
FROM ranked
WHERE r=1
ORDER BY client_id;
```

Actual execution: 40 rows; median 0.433 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

First row: client_id=20001; job_count=25

Q 12 Strictly overdue unfinished milestones as of execution date

Expected: Active contracts with unfinished milestones strictly overdue as of execution.

```
SELECT c.contract_id,
       COUNT(m.milestone_id) AS overdue_work
FROM contracts AS c
JOIN milestones AS m ON m.contract_id=c.contract_id
WHERE c.status='ACTIVE'
    AND m.status IN ('PENDING',
                     'IN_PROGRESS',
                     'COMPLETED')
    AND m.due_date < CURRENT_DATE
GROUP BY c.contract_id
ORDER BY c.contract_id;
```

Actual execution: 0 rows; median 0.219 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

Actual result set is empty on the tested state; no output row is invented.

Q 13 Rated freelancers ordered by average and review count

Expected: Rated freelancers ordered by mean and count.

```
SELECT u.full_name,r.* FROM v_freelancer_ratings AS r
JOIN users AS u ON u.user_id=r.freelancer_id
WHERE r.rating_count>0 ORDER BY r.average_rating DESC,r.rating_count DESC,r.freelancer_id;
```

Actual execution: 41 rows; median 1.140 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

First row: full_name=Đỗ Khánh Hà; freelancer_id=4; rating_count=1; average_rating=5.00

Q 14 Review distribution by star value

Expected: Number of submitted reviews at each present score.

```
SELECT rating,COUNT(*) AS review_count FROM contract_reviews GROUP BY rating ORDER BY rating;
```

Actual execution: 3 rows; median 0.202 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

First row: rating=3; review_count=40

Q 15 Successful revenue compared with rated contract count without fan-out

Expected: One row per freelancer with rating statistics and successful revenue joined after aggregation.

```
WITH revenue AS (
    SELECT payee_id,SUM(amount) AS paid_amount FROM v_payment_details
    WHERE payment_status='PAID' GROUP BY payee_id
)
SELECT r.freelancer_id,r.rating_count,r.average_rating,COALESCE(p.paid_amount,0) AS paid_amount
FROM v_freelancer_ratings AS r LEFT JOIN revenue AS p ON p.payee_id=r.freelancer_id
ORDER BY r.freelancer_id;
```

Actual execution: 205 rows; median 2.785 ms. Complete column names, five-row preview and plan are included in the evidence catalog.

First row: freelancer_id=4; rating_count=1; average_rating=5.00; paid_amount=500.00

7 Verification and security

7 1 Execution environment and scope

Fresh installation and the revised verification suite ran on MySQL Community Server 26.7.0, InnoDB and READ COMMITTED on Windows, through a task-owned data directory and loopback port 33370. Execution timestamp in UTC: 2026-10-20T11:41:33.989596+00:00. The original database service and source files were not used as destructive test targets. The isolated server was stopped after verification.

The suite recorded 59 passed checks and 0 failed checks. The previous package's 72 integrity, 20 RBAC and 12 query assertions are historical evidence of a different schema; none are counted toward this result. The new suite directly checks workflow outcomes, expected SQL errors, aggregate results, dataset invariants, a concurrent allocation and HTTP integration. It does not prove every possible schedule or all engine versions.

7 2 Test results

Check	Expected	Actual	Result
subtype disjoint	error 1644	error 1644	PASS
subtype retained	error 1644	error 1644	PASS
user type immutable	error 1644	error 1644	PASS
duplicate email	error 1062	error 1062	PASS
invalid job category	error 1452	error 1452	PASS
invalid budget	error 3819	error 3819	PASS
duplicate proposal	error 1062	error 1062	PASS
invalid proposal amount	error 3819	error 3819	PASS
confirmation without selection	error 1644	error 1644	PASS
contract parties immutable	error 1644	error 1644	PASS
contract terms immutable	error 1644	error 1644	PASS
milestone overflow	error 1644	error 1644	PASS
milestone skip	error 1644	error 1644	PASS
started terms immutable	error 1644	error 1644	PASS
payment before approval	error 1644	error 1644	PASS
duplicate successful payment	error 1062	error 1062	PASS
failed payment retry	accepted	accepted	PASS
invalid payment method	error 3819	error 3819	PASS
terminal payment immutable	error 1644	error 1644	PASS
payment history retained	error 1644	error 1644	PASS
review before completion	error 1644	error 1644	PASS
incomplete contract cannot close	error 1644	error 1644	PASS
duplicate review	error 1062	error 1062	PASS
review immutable	error 1644	error 1644	PASS
review retained	error 1644	error 1644	PASS

Check	Expected	Actual	Result
rbac no direct write	error 1142	error 1142	PASS
rbac no password hash	error 1142	error 1142	PASS
rbac no record payment	error 1370	error 1370	PASS
actor spoofing	error 1644	error 1644	PASS
outside contract review	error 1644	error 1644	PASS
freelancer cannot review	error 1370	error 1370	PASS
null actor	error 1644	error 1644	PASS
mutual consent creates one contract	assertion true	true	PASS
freelancer cannot approve	error 1644	error 1644	PASS
rating above domain	error 3819	error 3819	PASS
rating below domain	error 3819	error 3819	PASS
end to end payment and review	assertion true	true	PASS
completed job synced	assertion true	true	PASS
normalized payment columns	assertion true	true	PASS
payment view derives transfer	assertion true	true	PASS
rating aggregate oracle	assertion true	true	PASS
unrated mean is null	assertion true	true	PASS
no payment fanout	assertion true	true	PASS
concurrent budget serialization	assertion true	true	PASS
concurrent budget invariant	assertion true	true	PASS
invariant total disjoint subtype	assertion true	true	PASS
invariant unique proposal pair	assertion true	true	PASS
invariant unique selected proposal	assertion true	true	PASS
invariant confirmed contract exists	assertion true	true	PASS
invariant contract parties match	assertion true	true	PASS
invariant milestone budget	assertion true	true	PASS
invariant payment requires approved	assertion true	true	PASS
invariant unique paid attempt	assertion true	true	PASS
invariant milestone date bounds	assertion true	true	PASS
invariant review completion	assertion true	true	PASS
all 15 queries execute	assertion true	true	PASS
http reads database	assertion true	true	PASS
http writes database	assertion true	true	PASS
http csrf rejected	assertion true	true	PASS

7 3 Roles and actor identity

Role	Permitted operations	Denied operations
market_client	Post job, select proposal, allocate and approve work, complete contract, review	Direct base-table DML; impersonation; payment recording
market_freelancer	Submit and confirm proposal, progress or complete own work	Client approval, client reviews, direct table DML
market_payment	Record simulated payment outcome for the owning client	Unrelated general DML
market_analyst	Read five reporting views	Base tables and credential hashes

Procedures use SQL SECURITY DEFINER, but actor checks compare the supplied actor to the authenticated connection username obtained with USER(). CURRENT_USER() inside a definer routine identifies the definer and would not identify the caller. Accounts follow names such as client_1 and freelancer_4; account creation and mapping are DBA responsibilities. This is a coursework native-account identity design, not an application user-registration service. The installation includes GRANT and a temporary SELECT privilege followed by REVOKE to demonstrate revocation.

Sample accounts are initially locked and contain no committed passwords. For the demo, a DBA assigns local secrets and unlocks only the required accounts. The application refuses root and establishes a database connection with the credentials supplied by that native actor. Selected demo actors also receive read-only reporting rights across clients; those aggregate views are not tenant-private. An administrative installer owns definer objects in the local test; production deployment should assign a dedicated definer account with reviewed privileges.

7 4 Security and deployment limits

The demo binds to 127.0.0.1, uses HTTP Basic authentication only on loopback, validates a form token, parameterizes procedure arguments, escapes rendered values and never writes passwords to evidence. It must not be exposed to a network in this form. Production use would require HTTPS, managed authentication and sessions, stronger account lifecycle controls, logging, rate limiting and reviewed deployment credentials. TLS transport to MySQL, at-rest encryption, encrypted backups, restore drills and remote availability were not validated in this project run.

8 Application integration and group defense

8 1 Running the local demo

app/server.py uses Python's standard HTTP server and PyMySQL. After installing the database on a dedicated instance and configuring actor passwords, set DB_HOST, DB_PORT and APP_PORT as environment variables and run the server. It does not automatically load .env files. Open the loopback URL and authenticate using a native client, freelancer or payment account. Use a separate browser profile or private window for each role so the browser does not reuse the previous Basic credentials.

The page provides fixed forms for job posting, proposal submission, selection, confirmation, milestone allocation and progression, payment recording, contract completion and review submission. Reporting views show the latest 25 IDs. The SQL guide provides proposal and milestone lookups when the relevant ID is not in a dashboard. Actions outside the actor role are rejected by MySQL; the UI does not bypass stored procedures.

The verification suite made actual HTTP requests: a client authenticated, received database-backed reports, posted a new job that was then found in MySQL, and received HTTP 403 for a missing form token. The complete job-to-review workflow was also executed through native-account procedure calls. These are programmatic integration tests; the group still needs to rehearse browser interaction and deliver the live presentation.

8 2 Demonstration sequence

1. Client 1 posts a new job and notes its job_id.
2. Freelancer 4 submits a proposal; record proposal_id from the supplied lookup query.
3. Client 1 selects the offer; show that a contract does not yet exist.
4. Freelancer 4 confirms; inspect the single contract and fixed participants.
5. Client 1 allocates the agreed value to one or more milestones.
6. Freelancer 4 marks work IN_PROGRESS and then COMPLETED.
7. Client 1 approves; demonstrate rejection of payment before this step on another milestone.
8. The payment operator records FAILED and then PAID; show retries without duplicate revenue.
9. Client 1 completes the fully allocated and paid contract, then submits a score.
10. Inspect the rating count and mean, and show rejection of duplicate or out-of-range reviews.

8 3 Defense and peer evaluation

Each member must be able to explain keys, joins, constraints and one transaction on the live database. Suggested rehearsal responsibilities are accounts and normalization, workflow and concurrency, and queries plus application integration. These are proposed rehearsal topics, not claims about who performed the historical work. The separate Vietnamese defense guide contains questions and blank contribution and peer-evaluation forms for the group to complete honestly.

Live presentation, individual SQL defense and peer evaluation cannot be completed by generating this report. The team must rehearse, confirm actual contributions and carry out those assessment activities. The stage weights are 20%, 20%, 30% and 30%; separately, the rubric allocates 70% to group work and 30% to individual defense and peer evaluation. These are distinct breakdowns in the supplied handout.

9 Conclusion and limitations

The revised package implements the scenario with typed accounts, mutually confirmed contracts, budgeted milestones, approved payment attempts and completed-contract ratings. Payment attempts no longer repeat milestone-determined participants and amount. The twelve declared business relations meet BCNF under the stated candidate keys and FD assumptions. Conditional unique indexes and transactional rules preserve workflow constraints that normalization alone cannot enforce.

Fresh installation produced 7,343 fictional rows, including 120 reviews. The revised suite passed 59 checks and executed fifteen advanced queries on MySQL 26.7.0. The local application demonstrated actual HTTP reads and a committed write. The test fixture, execution evidence and runnable code accompany the report.

Remaining limits are explicit: compatibility with MySQL 8.0 has not been execution-tested in this revision; only one concurrent budget schedule is covered; calibrated production performance, encryption at rest and backup recovery are unverified; external money movement is simulated; the local Basic-auth demo is not suitable for public deployment; and live defense and peer evaluation remain team activities.

References

[1] INT1313 Databases. Industrial Documentation Standards and Frameworks. Semester 1, 2026–2027, pp. 1–2. Supplied lecturer PDF.

[2] INT1313 Databases. Milestone Project Plan and Evaluation. Semester 1, 2026–2027, pp. 1–2. Supplied lecturer PDF.

[3] Project 3 Freelance Marketplace Ecosystem Mini-Upwork. Supplied scenario image, including multi-party transactions, milestone tracking and user-rating aggregation.

[4] Team G3. Freelance Marketplace Ecosystem. Existing report and SQL workspace supplied for revision. This source provides the inherited entities, BR-01 to BR-16 and implementation baseline.

[5] Oracle. MySQL 8.0 Reference Manual. CREATE TABLE Statement. https://dev.mysql.com/doc/refman/8.0/en/create-table.html. Accessed 20 October 2026.

[6] Oracle. MySQL 8.0 Reference Manual. Locking Reads. https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-reads.html. Accessed 20 October 2026.

[7] Oracle. MySQL 8.0 Reference Manual. CREATE PROCEDURE and CREATE FUNCTION Statements. https://dev.mysql.com/doc/mysql/8.0/en/create-procedure.html. Accessed 20 October 2026.

[8] Revised G3 execution evidence. evidence/dataset_counts.json, schema_metadata.json, verification_results.json and query_results.json. Generated by database/tests/verify_project.py on the documented local instance.

Appendix A Business rule traceability

Rule	Enforcement	Evidence or verification scope
BR-01	users PK	duplicate identity is structurally rejected
BR-02	user_type CHECK and subtype triggers	subtype_disjoint, subtype_retained, user_type_immutable
BR-03 04	jobs NOT NULL FKs	invalid_job_category
BR-05	freelancer_skills composite PK and FKs	inherited fixture and executed dictionary
BR-06 07 09	proposals NOT NULL FKs	fresh install and reporting queries
BR-08	UNIQUE(job_id,freelancer_id)	duplicate_proposal
BR-10	unique selected-job expression and job lock	invariant_unique_selected_proposal
BR-11	ordered transitions and atomic contract trigger	confirmation_without_selection, mutual_consent_creates_one_contract
BR-12	contract guard and immutable identities	contract_parties_immutable, invariant_contract_parties_match
BR-13 14	milestone FK, positive CHECK, parent lock and aggregate	milestone_overflow, concurrent_budget_serialization
BR-15	ordered milestone states and approval guard	milestone_skip, payment_before_approval, freelancer_cannot_approve
BR-16	normalized attempt FK, derived view, unique PAID expression	failed_payment_retry, duplicate_successful_payment, payment_view_derives_transfer
BR-17	review PK, completion trigger, score CHECK and actor procedure	review before completion, rating above domain, outside_contract_review
BR-18	AVG and COUNT over completed-contract reviews	rating_aggregate_oracle, unrated_mean_is_null

Appendix B Reproduction and artifact map

The submission directory contains database/ for SQL, the data dictionary and revised tests; app/ for the local demonstration; diagrams/ for editable Graphviz source, SVG and PNG; and evidence/ for new execution results. README.md gives the installation order, isolated testing procedure and local actor setup. Administrative review notes are kept outside the narrative of this report.

Use INSTALL_DATABASE.sql once on a fresh database instance or schema with the required administrative privileges. Do not also run SAMPLE_DATA.sql afterward. The data-only script is an alternative for an empty schema after modules 01–04. The verifier refuses --fresh-install if the schema already contains tables; running verification without that flag still inserts verification rows and changes test accounts, so it is also restricted to a disposable instance.

Reproduction commands, actor configuration and query lookup examples are included in the Vietnamese README and demo guide. No passwords, production credentials or claims of completed peer evaluation are embedded in this report.